

Improving GPGPU Performance via Cache Locality Aware Thread Block Scheduling

Li-Jhan Chen, Hsiang-Yun Cheng, Po-Han Wang, and Chia-Lin Yang 

Abstract—Modern GPGPUs support the concurrent execution of thousands of threads to provide an energy-efficient platform. However, the massive multi-threading of GPGPUs incurs serious cache contention, as the cache lines brought by one thread can easily be evicted by other threads in the small shared cache. In this paper, we propose a software-hardware cooperative approach that exploits the spatial locality among different thread blocks to better utilize the precious cache capacity. Through dynamic locality estimation and thread block scheduling, we can capture more performance improvement opportunities than prior work that only explores the spatial locality between consecutive thread blocks. Evaluations across diverse GPGPU applications show that, on average, our locality-aware scheduler provides 25 and 9 percent performance improvement over the commonly-employed round-robin scheduler and the state-of-the-art scheduler, respectively.

Index Terms—GPGPU, thread block scheduling, cache locality

1 INTRODUCTION

GENERAL Purpose Graphic Processing Units (GPGPUs) are becoming increasingly popular for variant classes of applications, such as image processing, physical simulation, and cloud computing, due to their immense computing capability. Modern GPGPUs allow thousands of threads to be executed in parallel. With vast multi-threading, GPGPUs can offer better performance and power efficiency than CPUs [1].

Despite the high theoretically achievable thread-level parallelism, GPGPU cores suffer from serious cache contention. In each stream multiprocessor (SM) of modern GPGPUs, thousands of threads share a small L1 cache, resulting in extremely small per-thread cache capacity [2]. For example, the per-thread L1 cache capacity in NVIDIA's Fermi is only about 32 bytes [3]. With limited cache size, data in the cache can easily be evicted before being reused, leading to serious performance loss.

Prior studies have proposed different thread scheduling and cache management policies to mitigate cache contention. In typical GPGPUs, threads are grouped into thread blocks (TBs) and threads within a TB are grouped into warps. Some researchers introduced methods to throttle the number of warps that can access cache concurrently [4], [5]. However, warp throttling leaves other resources, such as issue width and memory bandwidth, under-utilized. Thus, instead of throttling threads, other researchers designed methods to selectively bypass low-locality memory requests [6], [7]. Most of these prior approaches primarily focus on managing the thread execution and cache utilization within a SM, but the impact of thread-to-SM mapping on performance is seldom discussed.

In this work, we tackle the GPGPU cache contention issue from a different angle. The mapping between TBs and SMs determines how much locality can be exploited inside each SM, while the warp scheduling within a single SM can only leverage the available

locality distributed by the thread block scheduler. With larger design space, a locality-aware thread block scheduler can provide tremendous opportunities to increase cache hit rate and improve performance. A naive heuristic method has been proposed in [8], which simply assumes that scheduling each pair of consecutive TBs to the same SM can exploit spatial locality. This solution only works for row-major applications. In this paper, we propose a generic thread block scheduler to cover different types of access pattern. The key challenge is how to accurately quantify the block-level spatial locality. We tackle this challenge by adopting a software-hardware cooperative approach to capture the amount of shared cache lines between different TBs. Through dispatching TBs with spatial locality to the same SM, the cache hit rate of each SM can be improved.

This paper offers the following contributions:

- We propose the first software-hardware cooperative mechanism to estimate the spatial locality among different TBs at run-time. Different from prior heuristic approaches, we leverage GPGPU characteristics to enable precise definition of spatial locality.
- Our mechanism can capture the spatial locality in a broad range of GPGPU applications with diverse memory access behavior. The proposed approach is generic in that the access pattern does not need to be known in advance, as long as the application can be represented by kernels with structural data accesses.
- We develop a locality-aware thread block scheduler to maximize spatial locality in each SM. Evaluations show that our approach can dramatically reduce L1 cache miss rate and improve performance by up to 52 percent (average 25 percent) and 34 percent (average 9 percent) over the conventional round-robin scheduler and the state-of-the-art approach.

2 BACKGROUND & MOTIVATION

In this section, we first present the background of GPGPUs. We then discuss our motivation to design a locality-aware thread block scheduler that can benefit various types of applications.

2.1 GPGPU Architecture

Fig. 1 shows a typical GPGPU architecture. It consists of a thread block scheduler, multiple SMs, a shared L2 cache, and an off-chip DRAM. The thread block scheduler is responsible for distributing each individual TB to the target SM. Each SM is composed of warp schedulers, register files, a private L1 cache, a texture cache, and a shared scratchpad memory. During kernel execution, the thread block scheduler dispatches all the threads within a TB to the same SM. In the SM, each TB can be further partitioned into fixed-size (usually 32 threads) warps by warp schedulers, and these warps are executed in single-instruction-multiple-data (SIMD) fashion.

2.2 Motivation

GPGPUs support enormous multi-threading to achieve high throughput, but the limited per-thread cache capacity may incur serious cache contention. For example, in NVIDIA's Kepler [9], each SM has a 48 KB L1 data cache and supports 2048 concurrent threads. Compared to the 16 KB per-thread cache capacity in Intel's Skylake-CPU [10], each thread of GPGPU can utilize far less L1 cache capacity (i.e., only about 24 bytes per thread). Thus, innovative thread block scheduling policies are needed to effectively utilize the precious cache resource.

The contention of L1 cache capacity can be alleviated by allocating TBs with shared cache lines to the same SM. In conventional

- L.-J. Chen, P.-H. Wang, and C.-L. Yang are with the National Taiwan University, Taipei City 10617, Taiwan. E-mail: {r03922026, f96922002, yangc}@csie.ntu.edu.tw.
- H.-Y. Cheng is with the Academia Sinica, Taipei City 11529, Taiwan. E-mail: hycheng@citi.sinica.edu.tw.

Manuscript received 11 Sept. 2016; revised 24 Mar. 2017; accepted 31 Mar. 2017. Date of publication 11 Apr. 2017; date of current version 5 Jan. 2018.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/LCA.2017.2693371

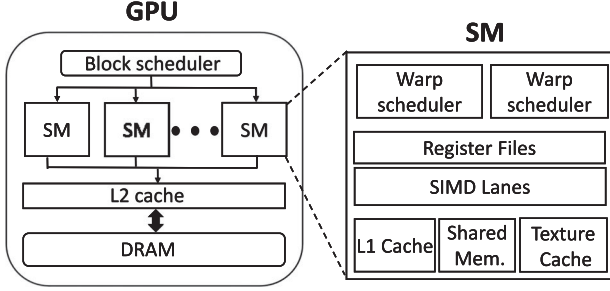


Fig. 1. The GPGPU architecture.

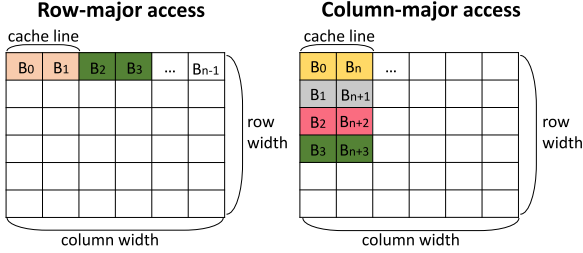


Fig. 2. Data locality in workloads with different access behavior.

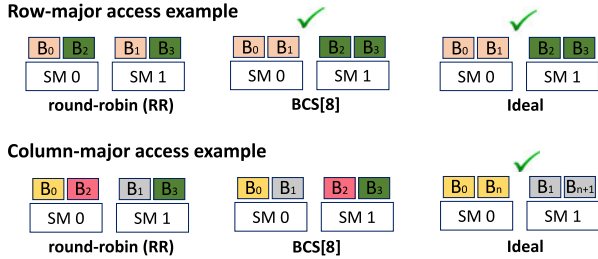


Fig. 3. Comparison of different thread block scheduling policies.

GPGPUs, the thread block scheduler is unaware of data locality and adopts a round-robin (RR) policy to dispatch TBs [11], [12]. Prior work [8] observed that consecutive TBs often share cache lines, so they proposed a scheduling policy (BCS) to group every two consecutive TBs into a pair and dispatch them to the same SM. Nevertheless, not all workloads benefit from BCS, as the TB combinations that share cache lines vary in workloads with different memory access behavior.

Fig. 2 shows two common types of memory access behavior in GPGPU workloads, and Fig. 3 illustrates the impact of different thread block scheduling policies on these two types of applications. TB i is denoted by B_i , and TBs that share cache lines are marked by the same color. Fig. 2 shows that consecutive TBs, such as $\{B_0, B_1\}$ and $\{B_2, B_3\}$, usually share cache lines in row-major applications. However, in column-major applications, cache lines are shared between stride TBs, such as $\{B_0, B_n\}$ and $\{B_1, B_{n+1}\}$, rather than consecutive TBs. In Fig. 3, we can observe that the commonly employed RR dispatches TBs with shared cache lines to different SMs, wasting the cache capacity. Prior BCS can only capture the performance improvement opportunities in row-major applications. As presented by the ideal case in Fig. 3, our goal is to design a locality-aware scheduler that can exploit the data reuse between TBs in different types of workloads, not limited to row-major or column-major applications.

3 LOCALITY-AWARE THREAD BLOCK SCHEDULER

In this paper, we propose a software-hardware cooperative approach to increase cache hit rate and improve performance by exploiting the spatial locality between different TBs. Fig. 4 shows the overview of the proposed scheme. We modify the compiler to

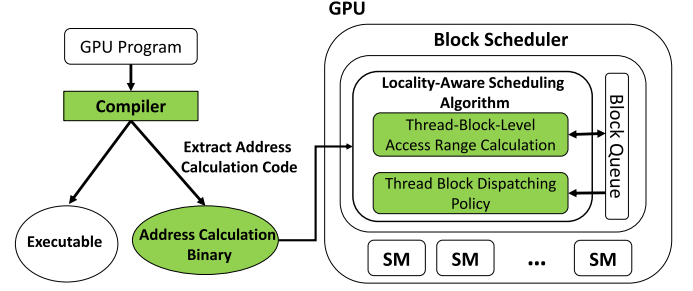


Fig. 4. Overview of the locality-aware block scheduler.

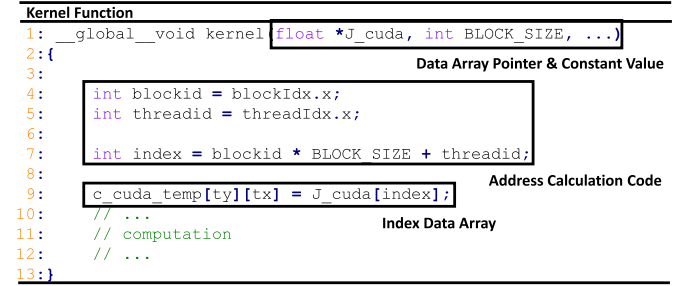


Fig. 5. Example of extracting the address calculation codes.

extract address calculation codes from kernel programs and generate a separate address calculation binary. The small, in-order CPU in modern GPU's thread block scheduler is utilized to run the locality-aware block scheduling algorithm, which is composed of two modules. The address range calculation module analyzes the memory access range of each TB when it is inserted into the block queue. To maximize the SM's inter-block spatial locality, the thread block dispatching module selects the candidate TB that should be scheduled next, and the selection process runs concurrently with thread block execution on SMs to hide the timing overhead.

3.1 Compiler Support

To capture the working sets of each TB, we modify the compiler to extract the address calculation code, so that the thread block scheduler can compute the memory access range based on the address calculation binary. A GPGPU program is composed of one or more kernels, and each kernel is an array of threads which run the same program code on different data. The mapping between thread IDs and data can be derived through simple arithmetics, since threads often operate on structural data, such as 1D or 2D arrays, in regular GPGPU programs. For instance, Fig. 5 shows a simplified kernel function, and the code segments that compute the mapping between thread ID and data are indicated by rectangular boxes. The compiler can easily extract the address calculation code, i.e., the code that is utilized to compute the index of the data array (lines 4-7), from a kernel function, and use the address calculation code and the base address of the data array to generate the address calculation binary. At run-time, the thread block scheduler can use the binary and the thread ID to compute the memory addresses accessed by an arbitrary thread.

3.2 Thread-Block-Level Access Range Calculation

The memory access range of each TB can be represented by a rectangle, since threads in regular GPGPU applications often access contiguous memory regions, such as linear 1D or 2D arrays. Fig. 6 shows an example of the rectangular thread-block-level access range. The address of the start point, i.e., the upper-left address, can be computed by the memory address of the first thread in the block, and the width/height can be computed by the address difference between the first and the last thread in the block. These

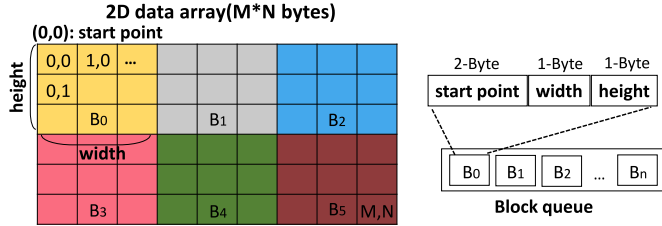


Fig. 6. Example of thread-block-level memory access range.

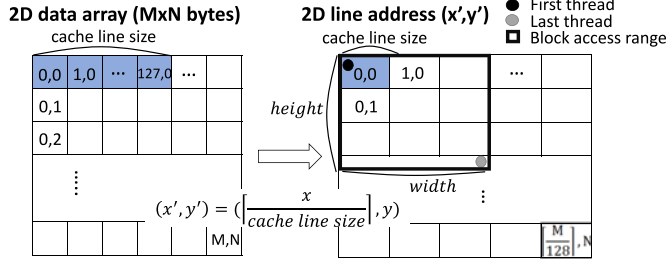


Fig. 7. The cache line level access range of a TB.

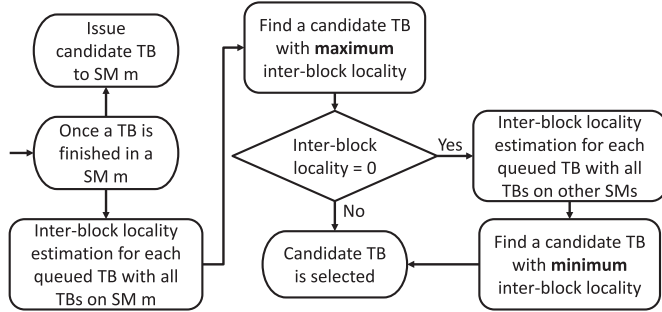


Fig. 8. Flow chart of the thread block dispatching policy.

access range parameters, including the start point, width, and height, are stored in the block queue when the block is inserted.

In order to compute the inter-block locality, we further use the coordination of cache lines to represent the access range rectangles. As shown in Fig. 7, the memory addresses of the data arrays can be transformed into the corresponding cache line addresses. For example, if a cache line is 128 bytes, memory addresses from (0,0) to (127,0) belong to the cache line (0,0) and memory addresses (128,1) to (255,1) belong to the cache line (1,1). Through the address transformation, the start point, width, and height of each TB can be represented in cache line coordination, as indicated by the bold rectangle.

3.3 Thread Block Dispatching Policy

Fig. 8 illustrates the flow chart of our locality-aware thread block dispatching policy. Once a TB is completed on a SM m , the thread block scheduler allocates the candidate TB to the SM. The selection process for the next candidate TB is then triggered and run concurrently with TB execution to hide the timing overhead. We select the candidate TB based on the overall inter-block locality (L_{all}), i.e., the summation of inter-block locality between the candidate TB and all the running TBs on the SM. The inter-block locality of any two TBs (L_{pair}) is defined as the summation of the overlapped data access range in all data arrays between them. For each data array, the overlapped area is computed by the following steps, as shown in Fig. 9:

- 1) $distance_x$ and $distance_y$ are the differences in x-axis and y-axis between the start points of two TBs.

Authorized licensed use limited to: Peking University. Downloaded on August 03, 2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

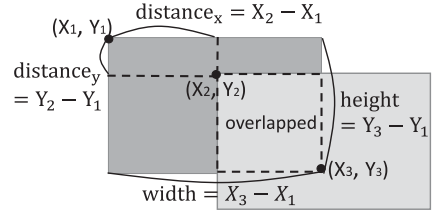


Fig. 9. Overlapped area between two thread blocks.

TABLE 1
System Configuration

Compute Units	15 SMs, 700MHz, SIMD width=32
Resources/SM	Max 1536 threads, Max 8 thread blocks 48 KB shared memory, 32768 registers
Warp Size	32 threads per warp
L1 Data Cache	16 KB per SM, 128 B line size, 4-way
L2 Cache	768 KB, 128B line size, 8-way
DRAM Model	GDDR5, FR-FCFS, 6 memory controllers

- 2) If $distance_x > width$ or $distance_y > height$, there is no overlapped area between the two TBs, indicating that there is no spatial locality between these two TBs.
- 3) Otherwise, the overlapped area is $(width - distance_x) * (height - distance_y)$, which is equal to the number of shared cache lines between these two TBs.

Based on the estimation of inter-block locality, the TB with maximum L_{all} is selected, as shown in Fig. 8. If no TB in the queue has inter-block locality on this SM, the TB with minimum L_{all} if being scheduled on other SMs is selected, so that the degradation of inter-block locality on other SMs can be avoided.

3.4 Overhead Analysis

Our locality-aware TB scheduler only incurs small hardware and timing overhead. To avoid duplicate calculation of memory access range, we add 4 bytes per block queue entry for storing related parameters, as shown in Fig. 6. In the TB scheduler, we add one register per SM to store the candidate TB, so that the candidate TB can be directly dispatched once a TB is completed on the SM. We use two methods to reduce the timing overhead of selecting candidate TBs. First, at the initial phase with less than 8 TBs being executed on the SM, we group consecutive TBs together and directly dispatch them to the same SM. After the initial phase, the TB selection process (Fig. 8) is executed concurrently with TB execution to hide the timing overhead.

4 EVALUATION METHODOLOGY

We evaluate our locality-aware thread block scheduler by modifying GPGPU-Sim [11]. Table 1 summarizes our system configuration. To show that our approach can benefit various applications, we use a combined set of workloads from two widely-used benchmark suites, Rodinia [13] and NVIDIA SDK [14], and use OpenCL to implement two well-known computer vision applications, SIFT and Gabor. These benchmarks are classified into three categories based on their memory access behavior, as shown in Table 2. Type-I applications are dominated by row-major accesses, while the memory accesses in Type-II applications are a mixture of different access patterns. For example, *SIFT* and *TRAN* have a mixture of row-major and column-major access pattern. One of the kernel in *LUD* has a mixture of row-major, column-major, and diagonal access pattern, so that different load instructions in a single TB share cache lines with different groups of TBs. Other workloads with no shared cache lines among TBs are Type-III. We compare our approach with conventional RR and the state-of-the-art BCS [8].

TABLE 2
Workload Classification

Type-I: row-major	
HotSpot [13] (HS)	SRAD [13]
Convolution filter [14] (CONV)	Histogram [14] (HIS)
Gabor filter (GF)	Back propagation [13] (BP)
Type-II: mixed access behavior (row-major, column-major, diagonal)	
LU decomposition [13] (LUD)	Matrix transpose [14] (TRAN)
Gaussian [13] (GAUS)	SIFT
Type-III: no shared cache lines	
Nearest Neighbor(NN) [13] (NN)	Needleman-Wunsch [13] (NW)

5 RESULTS

Fig. 10 shows the performance improvement of our locality-aware scheduler, and the L1 miss rates of different scheduling policies are shown in Fig. 11. Since our mechanism can dynamically capture the spatial locality among TBs for various applications, significant performance improvement is achieved. On average, our locality-aware scheduler can reduce L1 miss rate by 11 and 3 percent over RR and BCS, resulting in 25 and 9 percent performance improvement.

The amount of performance improvement is closely related to the memory access behavior of workloads. For row-major (Type-I) applications, our approach performs similarly as BCS. Since these workloads usually share cache lines between consecutive TBs, the thread block dispatching decisions made by our approach are similar to BCS. On average, our mechanism can slightly improve performance over BCS for Type-I applications. The small performance improvement over BCS comes from our ability to exploit the spatial locality among more than **two** consecutive TBs. For example, when running *HS* and *HIS*, the cache miss rate of our scheduler is lower than BCS. Our approach sometimes performs slightly worse than BCS, such as when running *SRAD*, because the benefits of thread block throttling in BCS outweigh its drawbacks in these rare cases. In these cases, the thread block throttling effect in BCS can slightly improve performance by reducing memory contention. For Type-II applications with mixed access behavior, our approach provides significantly better performance than RR and BCS, because our mechanism can dynamically capture the spatial locality that is not explored by BCS. On average, our thread block scheduler can provide 24 percent speedup over BCS in Type-II applications by reducing 8 percent cache miss rate. Note that our approach can significantly improve the performance of *LUD*, which has a mixture of row-major, column-major, and diagonal access behavior. For Type-III applications that have no shared cache lines among TBs, our scheduler does not provide benefits but there is no performance degradation.

6 RELATED WORK

Many prior studies designed warp scheduling policies to mitigate cache contention and minimize the memory access latency [4], [5], [15], [16]. For example, Rogers et al. [5] proposed a cache-conscious warp scheduling policy to improve intra-warp cache locality. They further combine a cache footprint predictor with the detection of

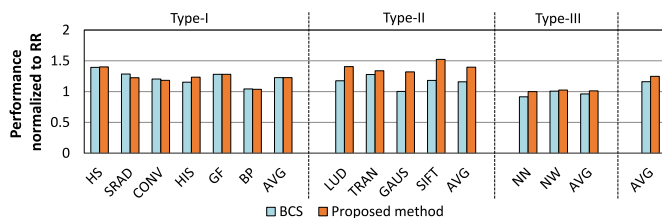


Fig. 10. Performance normalized to RR.

Authorized licensed use limited to: Peking University. Downloaded on August 03, 2026 at 08:20:24 UTC from IEEE Xplore. Restrictions apply.

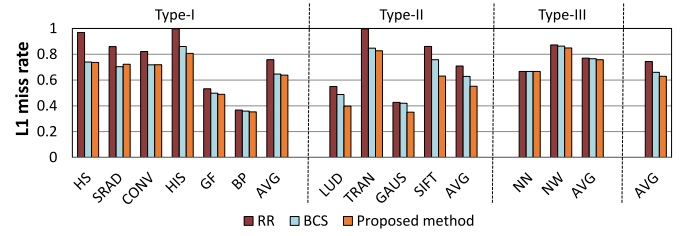


Fig. 11. Comparison of different policies in L1 cache miss rate.

static and dynamic memory access behavior to improve performance [4]. Jog et al. considers the impact of TB structure [15] and prefetching [16] to design warp scheduling policies for tolerating long memory access latency. Compared to warp scheduling policies, less prior work discussed how to design a better thread block scheduling policy for GPGPUs [8], [17], [18]. Similar to warp throttling techniques, Kayran et al. [17] proposed a dynamic TB scheduling policy to allocate fewer TBs for applications suffering from intensive memory contention. Wang et al. [18] proposed a TB scheduler specifically designed for dynamic parallelism execution models. They exploit the data locality between parent and child TBs to increase cache hit rates. Lee et al. proposed BCS [8], which statically allocates two consecutive TBs to the same SM, to increase the data reuse in cache. Nevertheless, BCS is only effective for applications with row-major access behavior. Our locality-aware block scheduler can estimate the spatial locality among TBs at runtime, resulting in performance improvement at a wide range of GPGPU applications.

7 CONCLUSION

In this paper, we propose a locality-aware thread block scheduler to improve GPGPU performance by reducing cache misses. We utilize a software-hardware cooperative method to capture the spatial locality among different TBs at runtime, and increase cache hit rates by dispatching TBs with numerous shared cache lines to the same SM. Results show that our approach can effectively reduce L1 miss rate (average 3 percent, up to 13 percent) and significantly improve performance (average 9 percent, up to 34 percent) over the state-of-the-art approach. Our work opens up new research directions for leveraging GPGPU software and hardware characteristics to precisely capture cache locality and optimize cache utilization. In the future, we will consider the impact of warp scheduling policies and explore the cooperation between block and warp schedulers to further improve cache locality.

ACKNOWLEDGMENTS

This work is supported in part by research grants from the Ministry of Science and Technology of Taiwan (MOST-105-2221-E-002-156-MY2, MOST-105-2622-8-002-002, and MOST-105-2218-E-002-025), and sponsored by MediaTek Inc., Hsin-chu, Taiwan.

REFERENCES

- [1] J. Nickolls and W. J. Dally, "The GPU computing era," *IEEE Micro*, vol. 30, no. 2, pp. 56–69, Mar./Apr. 2010.
- [2] W. Jia, K. A. Shaw, and M. Martonosi, "Characterizing and improving the use of demand-fetched caches in GPUs," in *Proc. 26th ACM Int. Conf. Supercomputing*, 2012, pp. 15–24.
- [3] NVIDIA, *NVIDIA's Next Generation CUDA Compute Architecture: Fermi*, Santa Clara, CA, USA, 2009.
- [4] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Divergence-aware warp scheduling," in *Proc. 46th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2013, pp. 99–110.
- [5] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *Proc. 45th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2012, pp. 72–83.
- [6] X. Xie, Y. Liang, Y. Wang, G. Sun, and T. Wang, "Coordinated static and dynamic cache bypassing for GPUs," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit.*, 2015, pp. 76–88.

- [7] D. Li, et al., "Priority-based cache allocation in throughput processors," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit.*, 2015, pp. 89–100.
- [8] M. Lee, et al., "Improving GPGPU resource utilization through alternative thread block scheduling," in *Proc. IEEE 20th Int. Symp. High Perform. Comput. Archit.*, 2014, pp. 260–271.
- [9] NVIDIA, *Kepler: The Fastest, Most Efficient HPC Architecture Ever Built*, Santa Clara, CA, USA, 2012.
- [10] Intel, *6th Generation Intel Core™ Processor Family Datasheet*, vol. 1, Santa Clara, CA, USA, 2016.
- [11] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2009, pp. 163–174.
- [12] K. M. Abdalla, L. V. Shah, J. F. Duluk, T. J. Purcell, T. Mandal, and G. Hirota, "Scheduling and execution of compute tasks," U.S. Patent US20130185725, Jul. 18, 2013.
- [13] S. Che, et al., "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE Int. Symp. Workload Characterization*, 2009, pp. 44–54.
- [14] NVIDIA, *CUDA C/C++ SDK Code Samples*, Santa Clara, CA, USA, 2011.
- [15] A. Jog, et al., "OWL: Cooperative thread array aware scheduling techniques for improving GPGPU performance," in *Proc. 18th Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2013, pp. 395–406.
- [16] A. Jog, et al., "Orchestrated scheduling and prefetching for GPGPUs," in *Proc. 40th Annu. Int. Symp. Comput. Archit.*, 2013, pp. 332–343.
- [17] O. Kayran, A. Jog, M. T. Kandemir, and C. R. Das, "Neither more nor less: Optimizing thread-level parallelism for GPGPUs," in *Proc. 22nd Int. Conf. Parallel Archit. Compilation Techn.*, 2013, pp. 157–166.
- [18] J. Wang, N. Rubin, A. Sidelnik, and S. Yalamanchili, "LaPerm: Locality aware scheduler for dynamic parallelism on GPUs," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit.*, 2016, pp. 583–595.